

APOTHECARY EXPRESS

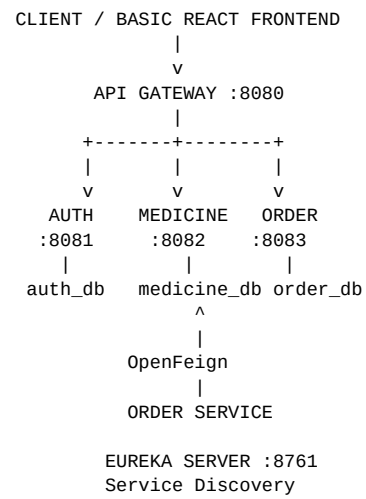
Backend-Focused Antigravity Master Prompt

Project: E-Pharmaceutical Supply Chain & Order Fulfillment Platform

Subject: SOA Programming and Microservices

Primary focus: Spring Boot Microservices Backend

Recommended Backend Architecture



The full Antigravity implementation prompt below prioritizes backend architecture, inventory consistency, JWT security, Eureka, Gateway, OpenFeign, PostgreSQL, Swagger, testing and a minimal frontend.

Paste-Ready Antigravity Master Prompt

You are a senior Java Spring Boot Microservices architect and developer.

Build the backend of an academic project called:

APOTHECARY EXPRESS

E-Pharmaceutical Supply Chain & Order Fulfillment Platform

SUBJECT:

SOA PROGRAMMING AND MICROSERVICES

The primary objective is to demonstrate a clean, understandable and functional MICROSERVICES BACKEND using Java, Spring Boot and Spring Cloud.

The backend is the MAIN focus of this project.

Do not over-engineer the system.

Do not add technologies that are not required by the problem statement.

=====

1. TECHNOLOGY STACK

=====

Use:

- Java 21
- Spring Boot 3.x
- Maven
- Spring Cloud
- Spring Cloud Gateway
- Netflix Eureka
- Spring Security
- JWT
- Spring Data JPA
- Hibernate
- PostgreSQL
- OpenFeign
- REST APIs
- Swagger / OpenAPI
- Jakarta Bean Validation

Frontend is secondary and should only be a simple client for testing the backend APIs.

=====

2. REQUIRED MICROSERVICES

=====

Create the following independent Spring Boot applications:

1. apothecary-eureka-server
2. apothecary-api-gateway
3. apothecary-auth-service
4. apothecary-medicine-service
5. apothecary-order-service

The final Eclipse / STS Package Explorer should look approximately like:

apothecary-express-parent

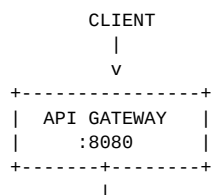
```
|
|-- apothecary-eureka-server
|-- apothecary-api-gateway
|-- apothecary-auth-service
|-- apothecary-medicine-service
|-- apothecary-order-service
```

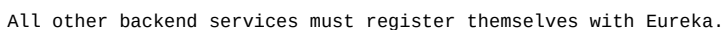
Each [boot] project must be an independently runnable Spring Boot application.

=====

3. HIGH LEVEL ARCHITECTURE

=====





Expected registered services:

AUTH-SERVICE
MEDICINE-SERVICE
ORDER-SERVICE
API-GATEWAY

Use service discovery instead of hard-coding service hostnames.

Example:

lb://AUTH-SERVICE
lb://MEDICINE-SERVICE
lb://ORDER-SERVICE

=====

6. API GATEWAY

=====

Create:

apothecary-api-gateway

Port:

8080

Routes:

/api/auth/**	-> AUTH-SERVICE
/api/users/**	-> AUTH-SERVICE
/api/medicines/**	-> MEDICINE-SERVICE
/api/orders/**	-> ORDER-SERVICE

Use Eureka service discovery.

Forward Authorization: Bearer JWT headers.

Keep gateway configuration simple and understandable.

=====

7. AUTH SERVICE

=====

Create:

apothecary-auth-service

Port:

8081

Database:

PostgreSQL

Entity:

User

Fields:

id
name
email
password
role
createdAt

Roles:

USER
ADMIN

Endpoints:

POST /api/auth/register
POST /api/auth/login
GET /api/users/me

Registration:

- Validate name

- Validate email
- Validate password
- Check duplicate email
- Hash password using BCrypt
- Save user

Login:

- Validate credentials
- Generate JWT
- Return JWT

JWT should contain:

- subject / user ID
- email if useful
- role
- issued time
- expiration time

NEVER store passwords in plain text.
NEVER place passwords inside JWT.

8. MEDICINE SERVICE

Create:

apothecary-medicine-service

Port:

8082

Database:

PostgreSQL

Medicine entity:

Medicine

Fields:

medicineId
name
category
price
stockQuantity
prescriptionRequired
createdAt
updatedAt

Use BigDecimal for price.

Do NOT use double or float for money.

9. MEDICINE APIs

PUBLIC:

GET /api/medicines
GET /api/medicines/{id}

ADMIN:

POST /api/medicines
PUT /api/medicines/{id}
PATCH /api/medicines/{id}/stock

INTERNAL ORDER WORKFLOW:

POST /api/medicines/{id}/reserve
POST /api/medicines/{id}/restore

The reserve and restore operations are intended for
inter-service communication.

10. CRITICAL INVENTORY REQUIREMENT

THIS IS THE MOST IMPORTANT BUSINESS RULE.

The system must NEVER allow stock to become negative.

Do NOT implement inventory like this:

1. SELECT stock
2. Check stock >= requested quantity
3. stock = stock - requested quantity
4. SAVE

This approach is vulnerable to concurrent orders.

Instead use an atomic conditional database update.

Conceptually:

```
UPDATE medicine
SET stock_quantity = stock_quantity - :quantity
WHERE medicine_id = :medicineId
AND stock_quantity >= :quantity;
```

Then check the number of affected rows.

If affected rows = 1:
Stock successfully reserved.

If affected rows = 0:
Return HTTP 409 CONFLICT.

Reason:
INSUFFICIENT_STOCK

This requirement MUST be implemented in the Medicine Service.

Use @Transactional where appropriate.

```
=====
11. ORDER SERVICE
=====
```

Create:

apothecary-order-service

Port:

8083

Database:

PostgreSQL

Entities:

Order
OrderItem

Order fields:

orderId
userId
totalAmount
orderStatus
createdAt
updatedAt

OrderItem fields:

orderItemId
orderId
medicineId
medicineName
unitPrice
quantity
subtotal

Use BigDecimal for:

price
subtotal
totalAmount

```
=====
```

12. ORDER STATUS

Use enum:

PENDING
CONFIRMED
PROCESSING
SHIPPED
DELIVERED
CANCELLED

Do not use random string values throughout the code.

13. ORDER APIS

POST /api/orders
GET /api/orders/{id}
GET /api/orders/my
PATCH /api/orders/{id}/status

14. ORDER PLACEMENT WORKFLOW

```
graph TD
    CLIENT --> V1((v))
    V1 --> API_GATEWAY[API GATEWAY]
    API_GATEWAY --> V2((v))
    V2 --> ORDER_SERVICE[ORDER SERVICE]
    ORDER_SERVICE --> V3((v))
    V3 --> MEDICINE_SERVICE[MEDICINE SERVICE]
    MEDICINE_SERVICE --> V4((v))
    V4 --> STOCK_SUCCESSFULLY[Stock successfully reduced]
    STOCK_SUCCESSFULLY --> V5((v))
    V5 --> ORDER_SERVICE2[ORDER SERVICE]
    ORDER_SERVICE2 --> V6((v))
    V6 --> CONFIRMED_ORDER[CONFIRMED ORDER]
```

CLIENT
|
v
API GATEWAY
|
v
ORDER SERVICE
|
| OpenFeign
v
MEDICINE SERVICE
|
| Check medicine
| Check stock
| Reserve stock
|
v
Stock successfully reduced
|
v
ORDER SERVICE
|
| Calculate total
| Save order
|
v
CONFIRMED ORDER

Example:

User orders:
Paracetamol x 2

Medicine Service:
stock = 10

After reservation:
stock = 8

Order Service:
unit price = 50
quantity = 2
total = 100

Order status:
CONFIRMED

15. DO NOT TRUST CLIENT TOTAL

The frontend must NOT be allowed to decide:

totalAmount
unitPrice

subtotal

The client should send only:

medicineId
quantity

Order Service must obtain authoritative medicine information from Medicine Service.

Calculate:

subtotal = unitPrice * quantity
totalAmount = sum(subtotals)

=====

16. OPENFEIGN

=====

Use Spring Cloud OpenFeign for:

ORDER SERVICE
|
v
MEDICINE SERVICE

Create a MedicineClient interface.

Conceptual operations:

getMedicine(id)
reserveStock(id, quantity)
restoreStock(id, quantity)

Use Eureka service discovery.

Do not hard-code:
http://localhost:8082

inside business logic.

Use:
MEDICINE-SERVICE

through service discovery.

=====

17. MULTI-ITEM ORDER FAILURE

=====

Example:

Medicine A x 2
Medicine B x 5
Medicine C x 1

Suppose:
A succeeds
B succeeds
C fails because stock is insufficient.

Do not leave A and B permanently deducted.

Implement simple compensation:

If a later medicine reservation fails:
restore previously reserved medicines.

Then reject the entire order.

Return:
HTTP 409 CONFLICT

Do NOT implement Kafka or distributed transactions.

=====

18. JWT SECURITY

=====

Use:

Spring Security

JWT
BCrypt

Public:

POST /api/auth/register
POST /api/auth/login

Medicine listing may be public:

GET /api/medicines
GET /api/medicines/{id}

Protected:

POST /api/orders
GET /api/orders/{id}
GET /api/orders/my

ADMIN only:

POST /api/medicines
PUT /api/medicines/{id}
PATCH /api/medicines/{id}/stock
PATCH /api/orders/{id}/status

=====

19. ORDER OWNERSHIP

=====

Mandatory:

User A must NOT access User B's order.

For:

GET /api/orders/{id}

verify:
authenticatedUserId == order.userId

unless authenticated user is ADMIN.

Return:
403 FORBIDDEN

Enforce this in backend code.

=====

20. DATABASE ARCHITECTURE

=====

Use database-per-service logically.

AUTH SERVICE:
auth_db

MEDICINE SERVICE:
medicine_db

ORDER SERVICE:
order_db

Each service owns its own tables.

Auth Service must not access medicine_db.

Order Service must not access medicine_db.

Order Service uses REST/OpenFeign.

Medicine Service owns inventory.

=====

21. SERVICE CODE STRUCTURE

=====

Each business service should use:

controller/
service/
repository/

entity/
dto/
exception/
config/

Example:

medicine-service

src/main/java/com/apothecary/medicine/

controller/
 MedicineController.java

service/
 MedicineService.java

repository/
 MedicineRepository.java

entity/
 Medicine.java

dto/
 MedicineRequest.java
 MedicineResponse.java
 StockUpdateRequest.java

exception/
 MedicineNotFoundException.java
 InsufficientStockException.java

config/
 SecurityConfig.java

Rules:

- Controllers stay thin.
- Business logic belongs in services.
- Database operations belong in repositories.
- Do not expose JPA entities directly.
- Use DTOs.
- Use constructor injection.

22. VALIDATION

Use Jakarta Validation:

@NotBlank
@Email
@NotNull
@Positive
@Min(1)

Validate:
medicine name
category
price
stock quantity
order quantity
email
password

23. GLOBAL EXCEPTION HANDLING

Use @RestControllerAdvice.

Handle:

ResourceNotFound
InsufficientStock
DuplicateEmail
InvalidCredentials
UnauthorizedOrderAccess
Validation errors
Downstream service failures

Use:

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
500 Internal Server Error

Example:

```
{
  "timestamp": "...",
  "status": 409,
  "error": "INSUFFICIENT_STOCK",
  "message": "Requested quantity is greater than available stock",
  "path": "/api/orders"
}
```

24. SWAGGER / OPENAPI

Add Swagger/OpenAPI to:

AUTH SERVICE
MEDICINE SERVICE
ORDER SERVICE

Document:

- endpoints
- request DTOs
- response DTOs
- error responses
- JWT Bearer authentication

Swagger must be usable for backend demonstration without frontend.

25. CONFIGURATION

Use application.yml or application.properties.

Use environment variables where appropriate:

DATABASE_URL
DATABASE_USERNAME
DATABASE_PASSWORD
JWT_SECRET

Do not hard-code real secrets.

26. PORTS

Eureka Server: 8761
API Gateway: 8080
Auth Service: 8081
Medicine Service: 8082
Order Service: 8083

27. TESTING

Create tests for:

AUTH:

- successful registration
- duplicate email
- successful login
- invalid credentials

MEDICINE:

- medicine creation
- medicine retrieval
- stock update
- insufficient stock
- successful stock reservation

ORDER:

- successful order
- invalid quantity
- insufficient stock
- total calculation
- order ownership
- admin status update

CRITICAL:

Test concurrent order attempts and verify:
stock never becomes negative.

=====
28. DATABASE SEED DATA
=====

Create fictional demo medicines:

Paracetamol
Pain Relief
50
100

Amoxicillin
Antibiotic
120
50

Cetirizine
Allergy
30
75

Create demo ADMIN and USER accounts.

No real patient data.

=====
29. FRONTEND
=====

Frontend is secondary.

Create only a basic React/Vite client.

Required screens:

Login
Register
Medicines
Cart
Orders
Admin

Frontend communicates ONLY with:

<http://localhost:8080>

Do not spend excessive time on styling.

Backend correctness has priority.

=====
30. DO NOT OVERENGINEER
=====

DO NOT add:

Kafka
Redis
Kubernetes
Docker Swarm
GraphQL
CQRS
Event Sourcing
ELK
RabbitMQ
WebSockets
Service Mesh

Focus on:

Spring Boot

Microservices
REST
Eureka
API Gateway
JWT
PostgreSQL
OpenFeign
Swagger
Inventory
Orders

=====

31. DEVELOPMENT ORDER

=====

Build in this exact order:

1. Parent Maven project
2. Eureka Server
3. Auth Service
4. Medicine Service
5. Order Service
6. API Gateway
7. JWT security integration
8. OpenFeign communication
9. Swagger
10. Tests
11. Basic frontend
12. Final integration testing
13. README

After each service:

- compile
- test
- verify configuration
- verify database
- verify Eureka registration
- verify APIs

Do not generate fake implementations just to make compilation pass.

=====

32. EXISTING REPOSITORY RULES

=====

Before changing anything:

1. Inspect the repository.
2. Identify existing modules.
3. Identify pom.xml files.
4. Identify application.yml/properties.
5. Reuse correct existing configuration.
6. Do not delete working code blindly.
7. Do not create duplicate modules.
8. Do not create duplicate entities.
9. Do not create duplicate security configurations.

If unrelated microservices already exist, do not mix them into Apothecary Express.

=====

33. CODE QUALITY

=====

Use:

- Constructor injection
- DTOs
- Service layer
- Repository layer
- Java enums
- Meaningful names
- Small methods
- Proper exception handling
- Transactions where required
- BigDecimal for money

Avoid:

- Field injection
- Giant controllers

- Giant services
- Static global state
- Hard-coded service URLs
- Hard-coded passwords
- Returning entities directly
- Catching Exception everywhere
- Duplicate business logic

34. FINAL SOA/MICROSERVICE REQUIREMENTS

The project must clearly demonstrate:

1. Independent services
2. Service discovery
3. API Gateway
4. REST APIs
5. Inter-service communication
6. JWT authentication
7. Role-based authorization
8. Database-per-service
9. Service ownership of data
10. Inventory consistency
11. Order workflow
12. Error handling
13. API documentation
14. Testing

35. FINAL DEMONSTRATION

Start:

Eureka
Gateway
Auth
Medicine
Order

Open:

http://localhost:8761

Show registered:

AUTH-SERVICE
MEDICINE-SERVICE
ORDER-SERVICE
API-GATEWAY

Then:

1. Register USER.
2. Login.
3. Receive JWT.
4. List medicines through Gateway.
5. Place an order.
6. Order Service calls Medicine Service through OpenFeign.
7. Medicine stock decreases.
8. Order becomes CONFIRMED.
9. Attempt order exceeding stock.
10. Receive HTTP 409.
11. Verify stock did not become negative.
12. Login as ADMIN.
13. Update medicine stock.
14. Update order status.

36. FINAL DELIVERABLE

At completion provide:

- A. Complete project tree
- B. Architecture diagram
- C. Service list
- D. Port numbers
- E. Database tables
- F. API endpoint list
- G. JWT flow

H. Eureka flow
I. Gateway flow
J. Order flow
K. Inventory reservation algorithm
L. OpenFeign communication
M. Swagger URLs
N. Test results
O. Startup commands
P. Sample API requests/responses
Q. README.md
R. Known limitations

=====

37. PRIORITY

=====

Priority:

1. BACKEND ARCHITECTURE
2. MICROSERVICE COMMUNICATION
3. INVENTORY CONSISTENCY
4. JWT SECURITY
5. DATABASE
6. ORDER WORKFLOW
7. EUREKA
8. API GATEWAY
9. SWAGGER
10. TESTING
11. BASIC FRONTEND

Do NOT sacrifice backend correctness for frontend appearance.

The final project should look like a proper Spring Boot microservices workspace in Eclipse/STS and should be straightforward for a faculty member to understand and demonstrate.

Start by inspecting the repository and provide a concise implementation plan before making major changes.

Expected Final Workspace

```
apothecary-express-parent
|
+-- apothecary-eureka-server      [boot]
+-- apothecary-api-gateway        [boot]
+-- apothecary-auth-service       [boot]
+-- apothecary-medicine-service   [boot]
+-- apothecary-order-service      [boot]
+-- apothecary-frontend
```